

PalmSource®

Conference and Expo

PalmSource®
Conference and Expo



Designing the Non-Simple Application

Neil Rhodes
Author and Contract Developer, Calliope Enterprises, Inc.



We are going to talk about how to use a toolchain (CodeWarrior or PRC-Tools) to build complex applications.



Agenda Slide

- Breaking the 64K barrier
- Static libraries
- Shared libraries
- Designing a plug-in architecture
- Q&A



Breaking the 64K Barrier

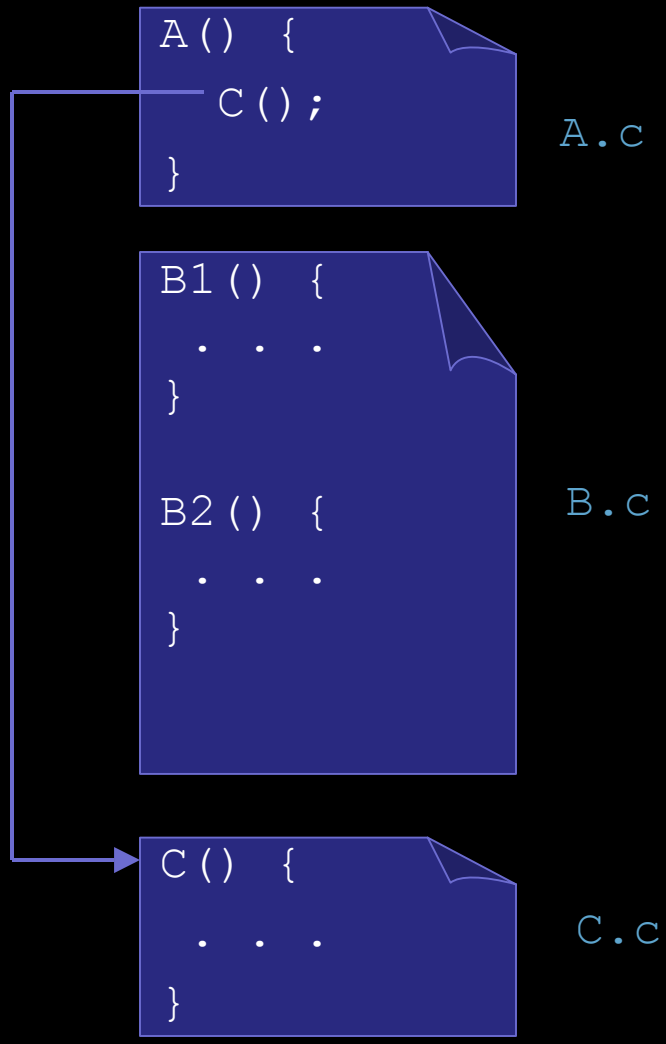
- 32K barrier
  - Subroutine calls are normally made with 16-bit signed offsets
- 64K barrier
  - Code is in ‘code’ resource
  - Resources can’t exceed 64K

Breaking the 32K Barrier

The Problem

- One function calls another more than 32K away

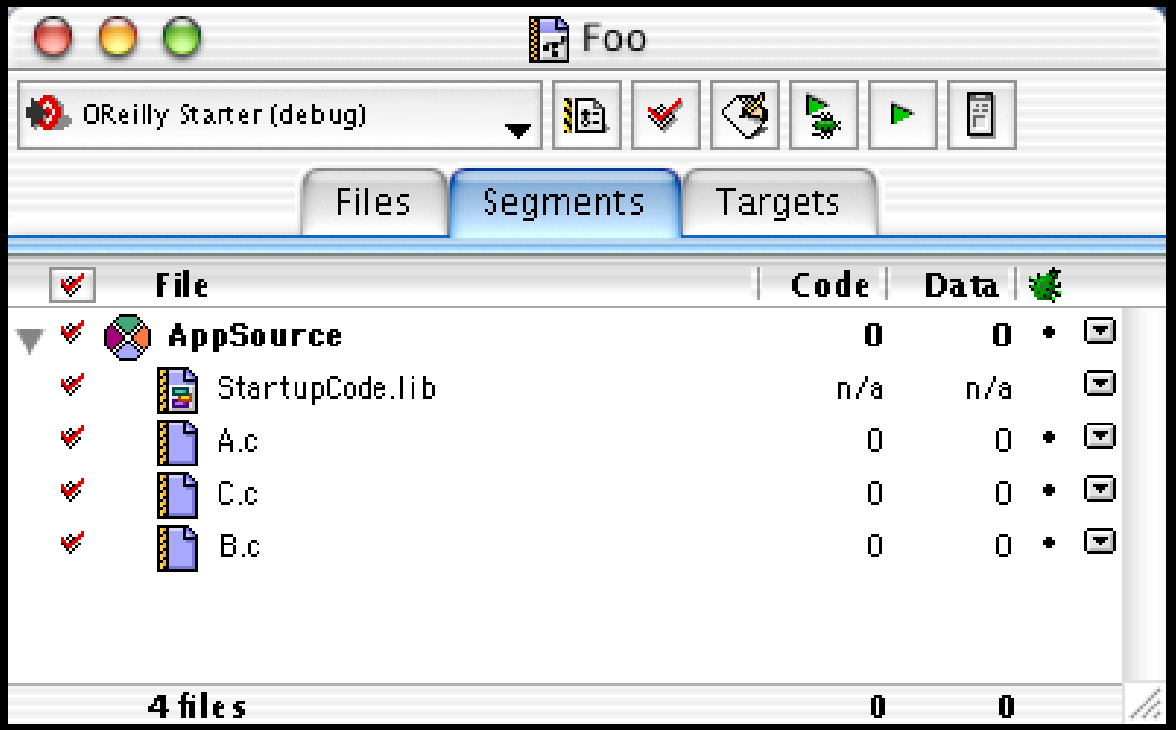
```
Link Error: A.c: 'A' 16-bit
 code reference to 'C' is
 out of range.
```



Breaking the 32K Barrier

Solution #1

- Change the link order




```
A() {
   C();
}
```

A.c

```
C() {
  . . .
}
```

C.c

```
B1() {
  . . .
}

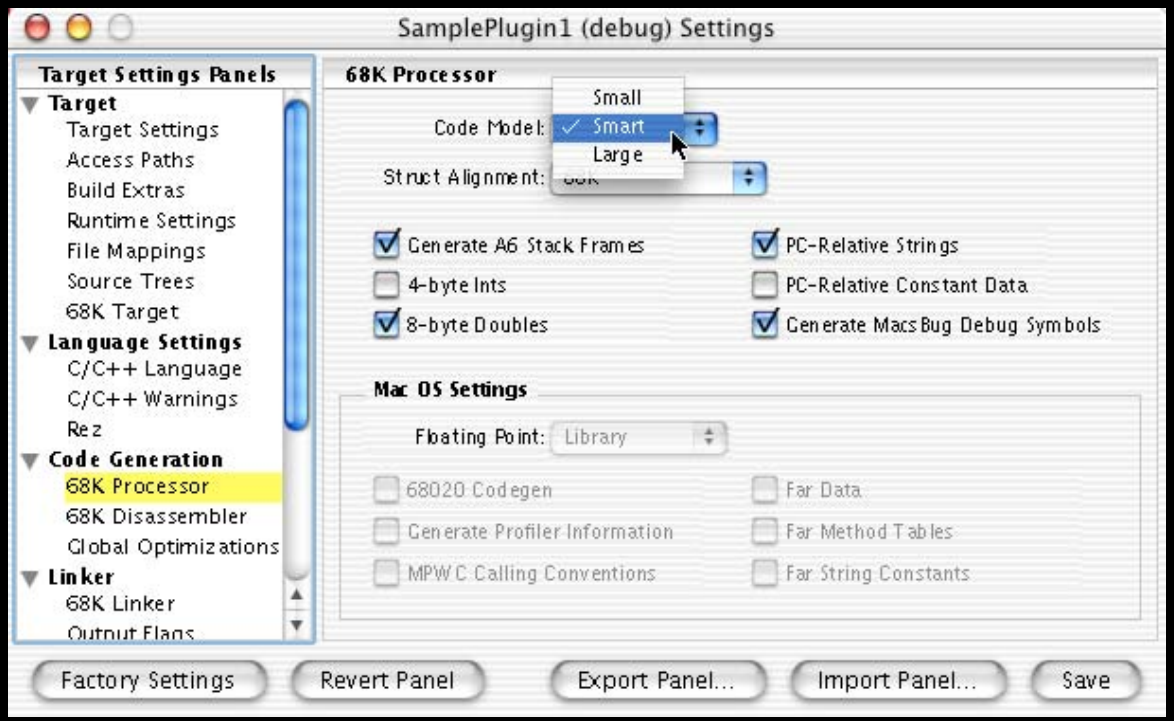
B2() {
  . . .
}
```

B.c

Breaking the 32K Barrier

Solution #2

- Use “Smart” code model (in CodeWarrior)



32-bit jump

```
A() {
   C();
}
```

A.c

```
B1() {
  . . .
}

B2() {
  . . .
}
```

B.c

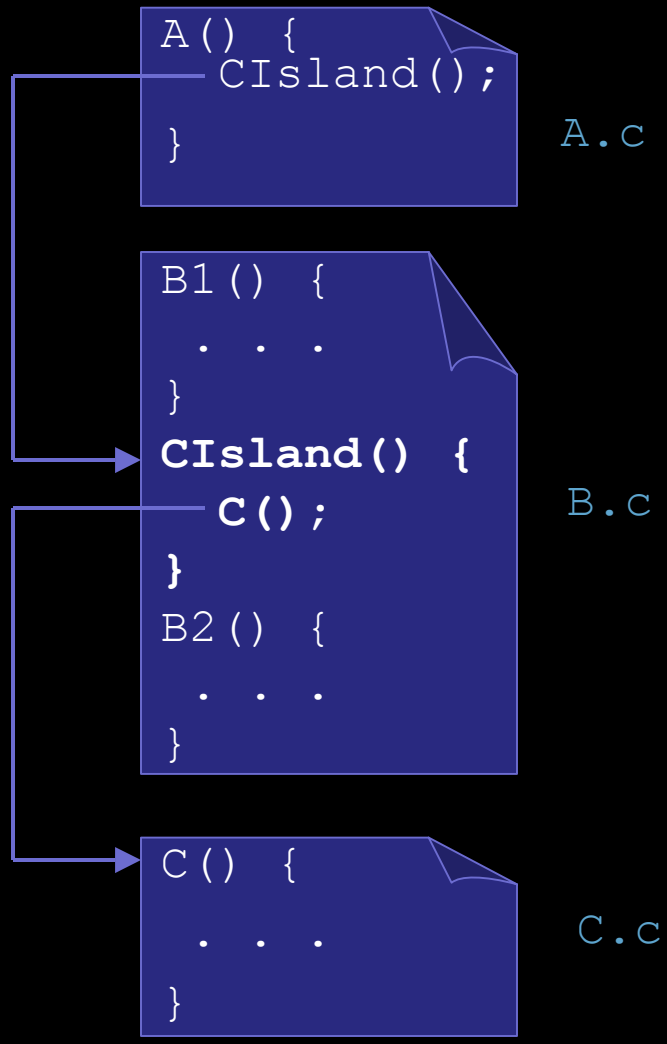
```
C() {
  . . .
}
```

C.c

Breaking the 32K Barrier

Solution #3

- Jump islands
- Disadvantages
  - Lots of manual work
  - Not very maintainable





Breaking the 64K Barrier

- Problem: total code exceeds 64K and won't fit in a resource
- Solution: Create a multi-segment application by breaking the code into multiple resources

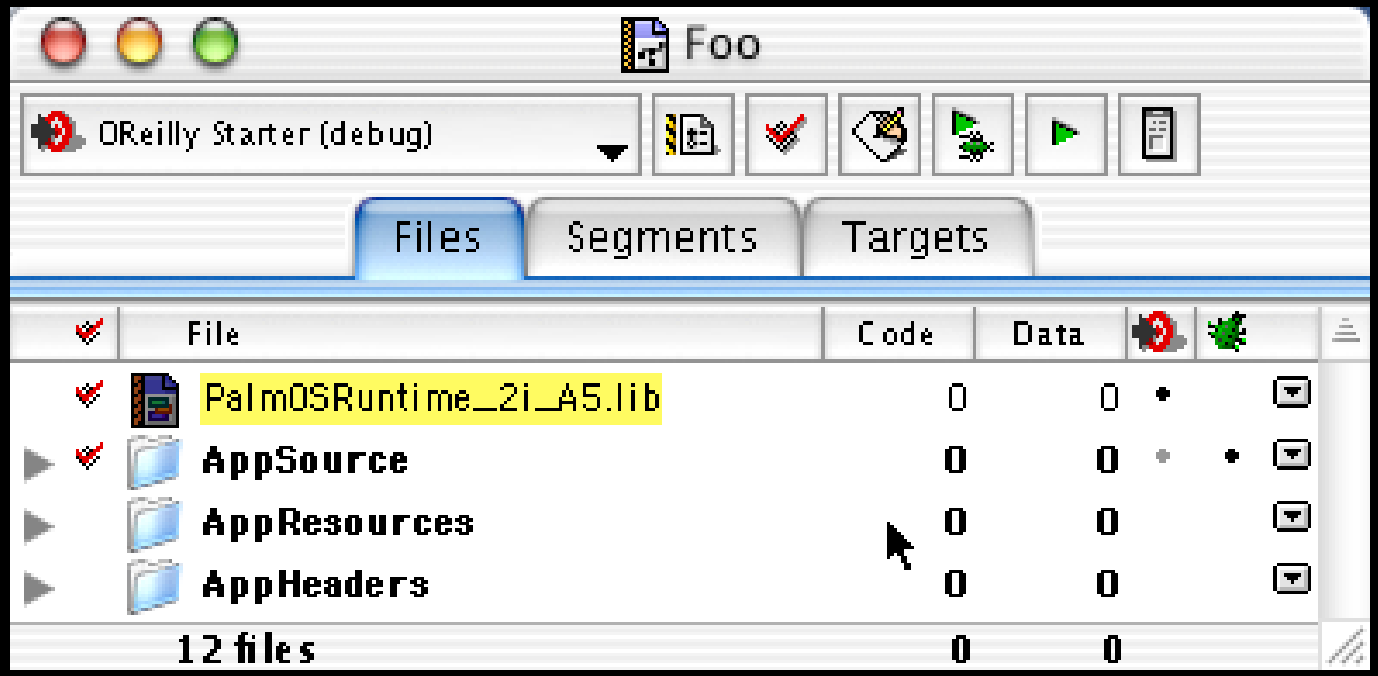


Breaking the 64K Barrier

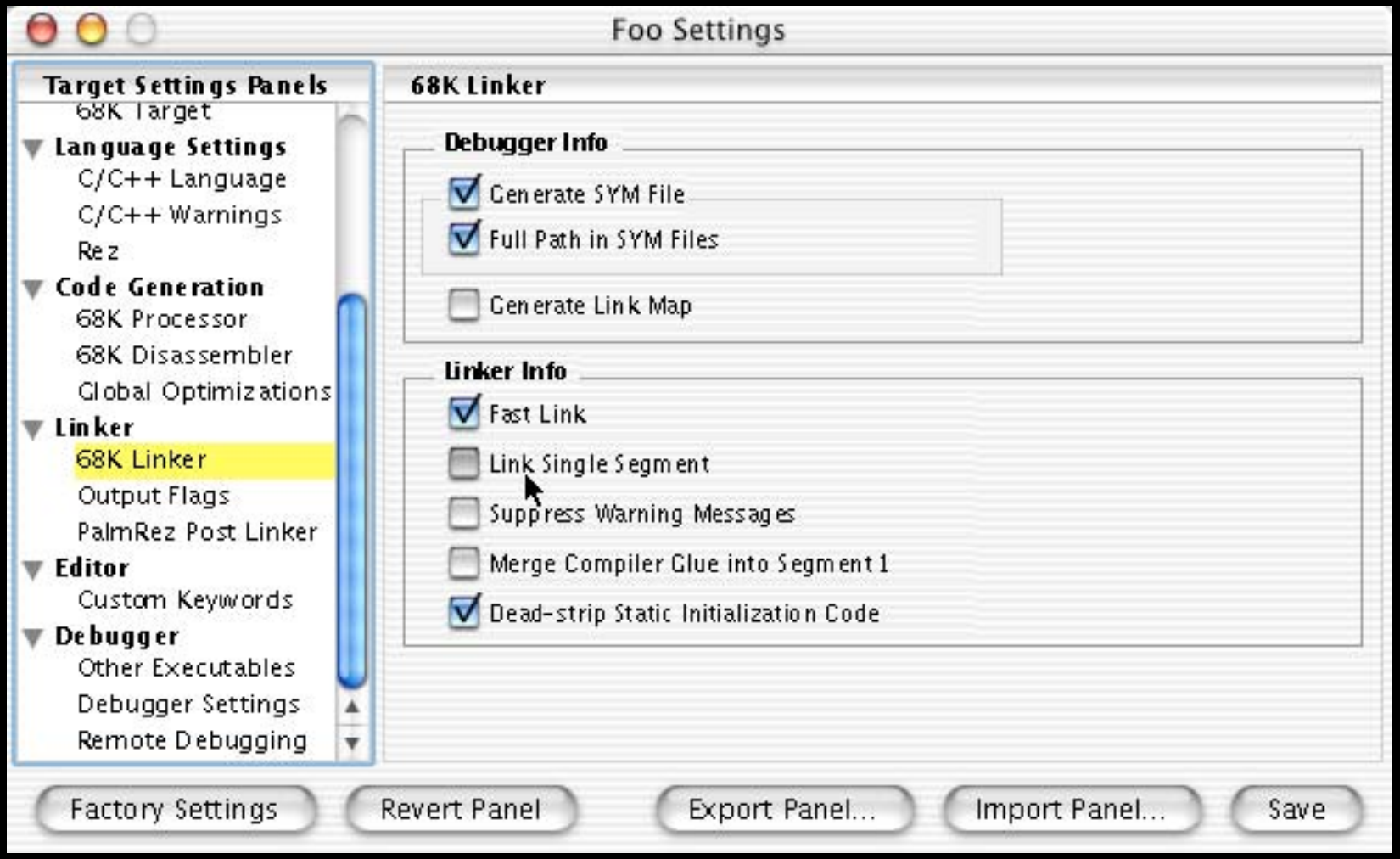
- Steps for CodeWarrior
  - Replace runtime library
  - Turn off “Link Single Segment”
  - Assign code to segments
- Steps for PRC-Tools
  - Assign code to segments in declarations
  - Specify segments in .def file
  - Adjust Makefile

Replace Runtime Library

- Replace “StartupCode.lib” with “PalmOSRuntime_2i_A5.lib”



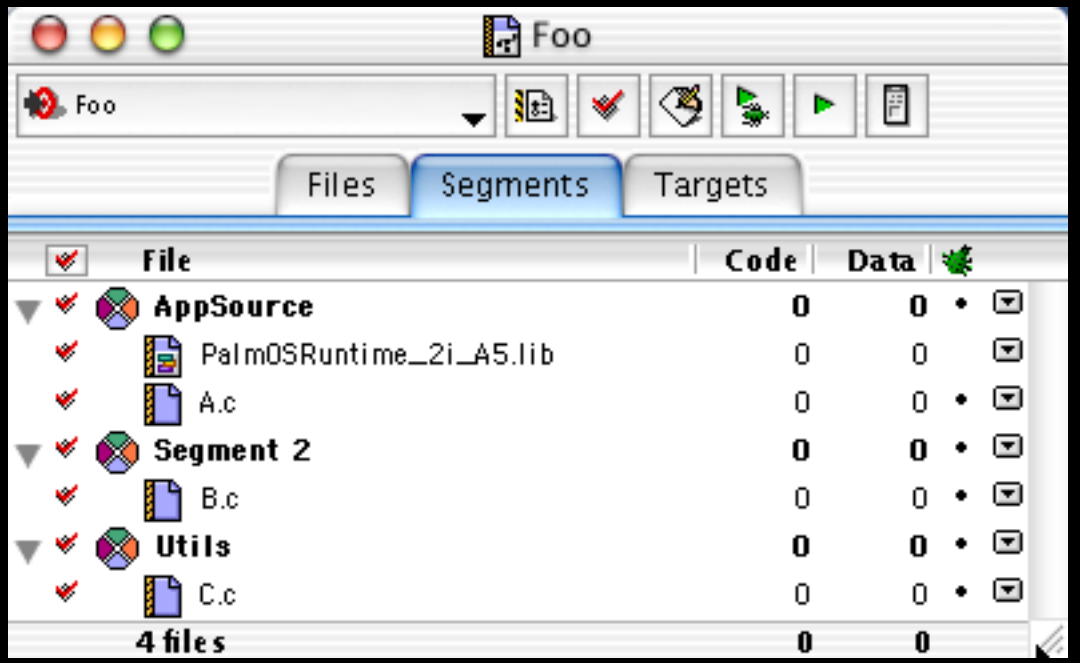
Turn off “Link Single Segment”



Assign Code to Segments

File-by-File Segmentation

- Use Segments panel





Assign Code to Segments

Function-by-Function Segmentation

- Use **`#pragma segment`**

```
#pragma segment DetailsForm
Boolean DetailsFormHandleEvent(…)
{
    …
}

#pragma segment Utils
Err DuplicateHandle(MemHandle *h)
{
    …
}
```

Multi-Segment Applications

- Inter-segment function calls are done via a global jump table
- No globals means no segments
- Must be in first segment:
  - PalmOSRuntime_2i_A5.lib
  - **`PilotMain`**
  - All code called by **`PilotMain`** when **`launchCode`** is not **`sysAppLaunchCmdNormalLaunch`** or **`sysAppLaunchCmdGoTo`**

Multi-Segment Application

- Comment out code in **`PilotMain`** for handling:
  - **`sysAppLaunchCmdNormalLaunch`**
  - **`sysAppLaunchCmdGoTo`**
- Turn on “Generate Link Map” (68K Linker panel)
- Build
- Every function mentioned in .MAP file must be in first segment



Segmenting with PRC-Tools

- In function declaration, add an attribute:

```
Boolean DetailsFormHandleEvent(...)
__attribute__((section ("DetailsForm")));

Err DuplicateHandle(MemHandle *h)
 __attribute__((section("Utils")));
```

- Declaration must be visible to both callers and function definition
- In .def file, add list of segments:

```
multiple code {"DetailsForm", "Utils"}
```



Agenda Slide

- Breaking the 64K barrier
- Static libraries
- Shared libraries
- Designing a plug-in architecture
- Q&A

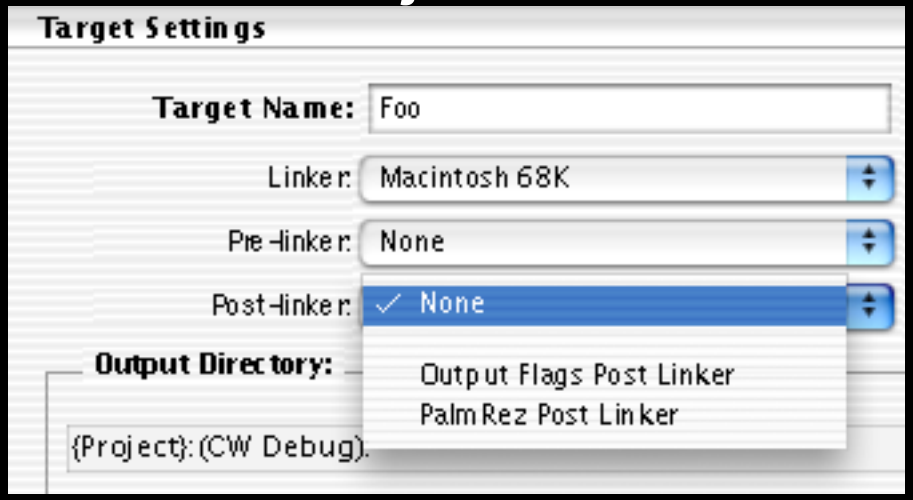


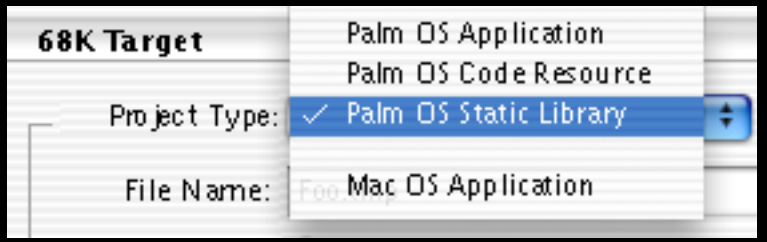
Libraries

- Static libraries
  - Linked with project at compile time
  - CW Linker only includes called functions
  - PRC-Tools linker includes entire object file
  - Must be included for each project that uses
  - Usually has header file for constants, prototypes, etc.
  - Easy to distribute without exposing source
- Shared libraries
  - Code resources on device
  - Can be called by other applications at runtime
  - Similar to a DLL in concept

Static Libraries—How-to with CodeWarrior

- Turn off Post linker
- Change the project type to “Palm OS® Static Library”
- Remove any runtime libraries or resources from the Project window







Static Libraries—How-to with PRC-Tools

- Use **`m68k-palmos-coff-ar`** command to create a .a file from multiple .o files
- Use **`m68k-palmos-coff-ranlib`** command to add an index to the .a file
- Link to the .a file



Agenda Slide

- Breaking the 64K barrier
- Static libraries
- Shared libraries
- Designing a plug-in architecture
- Q&A



Shared Libraries

- What a shared library is
- Why use a shared library
- How it is called



What a Shared Library Is

- Resource database of type ‘libr’
- Contains ‘libr’ (0) resource (<64K)
- Examples
  - Net Library
  - Internet Library
  - Exchange Manager
  - MathLib



Why use a Shared Library?

- Shared application services
  - Communications
  - Compression
  - Data Parsing
- Sleep/wake notification (not needed in OS 3.5 or later)
  - Hardware drivers
- Reduce application footprint
  - MathLib



How it is Called

- **`SysLibLoad(..., &refNum)`**
  - **`SysLibFind("libraryName", &refNum)`**
    - ***`LibName`*`Open(refNum, ...)`**
    - ***`LibName`*`Function1(refNum, ...)`**
    - ***`LibName`*`Function2(refNum, ...)`**
    - ***`LibName`*`Close(refNum, ...)`**
- **`SysLibRemove(refNum)`**

Loading a Shared Library

- **`SysLibLoad`**
  - calls shared library to obtain its dispatch table
  - stores dispatch table (and pointer to globals) in array of loaded libraries
- Library refNum is index into that array
- Most system libraries are automatically loaded on system boot
- Third-party libraries must be loaded explicitly
- To unload, call **`SysLibRemove`**

Finding a Shared Library

- **`SysLibFind`** returns a refnum to a loaded library with the given name

```
err = SysLibFind("MyLibrary",
  &refNum)
```



Dispatch Table

- Contains function pointers
- First four must be:
  - Open
  - Close
  - Sleep
  - Wake
- Additional functions are specific to the shared library

Calling a Shared Library Function

- Each shared library function is declared as a sys trap with a distinct trap number, with the refNum as the first parameter:

```
extern Err MyLibOpen(UInt16 refNum,UInt32
*contextP) SYS_TRAP(sysLibTrapOpen);

 extern Err MyLibFoo(UInt16 refNum,
   UInt32 *val) SYS_TRAP(MLTrapFoo));
```

- The trap uses:
  - The refNum to find the correct dispatch table
  - The trap number to find the correct function in the dispatch table



Shared Library Limitations

- No more than 2048 functions
- No global variables
  - Saved state can be maintained in a memory chunk
- Library may be opened more than once
  - Open and close must maintain a reference count



For More Information

- Knowledge Base
  - Article 1143—*Shared Libraries and other Advanced Project Types*
  - Article 1154—*SampleLib project*
  - Article 1670—*Mastering Shared Libraries*
- Internet
  - *Shared Libraries on the Palm OS®*—Ian Goldberg
    - www.isaac.cs.berkeley.edu/pilot/shlib.html



Agenda Slide

- Breaking the 64K barrier
- Static libraries
- Shared libraries
- Designing a plug-in architecture
- Q&A



Designing a Plug-in Architecture

Goals

- A main application that's extensible at run-time by plugins
- Plugins should be able to have globals and user interface

Designing a Plug-in Architecture

- Plugins are applications, but have a different type (for example, ‘PLUG’)
- Main application creates list of plugins using **`DmGetNextDatabaseByTypeCreator`**
- Main application launches a plugin using **`SysUIAppSwitch`** with:
  - Custom launch code (**`kLaunchPluginLaunchCode`**)
  - Custom launch parameters (creator ID and type of app to which to return)

Designing a Plug-in Architecture

- Plugin returns to main application using **`SysUIAppSwitch`** with:
  - Custom launch code (**`kReturnFromPluginLaunchCode`**)
  - Custom launch parameters (if needed)



Designing a Plug-in Architecture

- Sample available at ftp://ftp.calliopeinc.com/PluginArchitecture/



Agenda Slide

- Breaking the 64K barrier
- Static libraries
- Shared libraries
- Designing a plug-in architecture
- Q&A



PalmSource®

Conference and Expo